Análisis comparativo del rendimiento de los algoritmos Insertion Sort, Shell Sort y Quick Sort

Comparative analysis of the performance of the Insertion Sort, Shell Sort, and Quick Sort algorithms

Nombre: Jonathan Daniel Perca Mamani
Código: 2025-119031
Carrera: Ingenieria en Informatica y Sistemas (ESIS)
Facultad: Facultad de Ingeniería (FAIN),
Universidad: Jorge Basadre Grohmann (UNJBG),
Ciudad: Tacna
País: Perú
Correo: jpercam@unjbg.edu.pe

Resumen (ES): El propósito del estudio fue comparar el desempeño de los algoritmos Shell Sort, Insertion Sort y Quick Sort en función de distintos tamaños y patrones de entrada. Se implementaron tres algoritmos en C++ y se anotaron las comparaciones, las profundidades de recursión (cuando fueron necesarias), los movimientos y los tiempos de ejecución. Se tomaron en cuenta arreglos de 1 000, 10 000 y 100 000 elementos, con cinco patrones de entrada y un máximo de 30 repeticiones por condición experimental. Para un conjunto de 100 000 elementos, los resultados iniciales revelaron diferencias significativas entre los algoritmos. En las configuraciones de ordenamiento con duplicados, casi ordenado, descendente, y aleatorio, Quick Sort tuvo tiempos promedio más bajos que Shell Sort e Insertion Sort. Insertion Sort mostró el tiempo promedio más bajo en la secuencia ascendente. Se estableció de manera preliminar que el rendimiento observado estaba condicionado por el algoritmo y el patrón de entrada, lo cual indica que para hacer la comparación es necesario tener en cuenta las condiciones experimentales y la calidad de las mediciones.

Palabras clave: Algoritmos de ordenación, Insertion Sort, Shell Sort, Quick Sort, tiempo de ejecución, análisis comparativo.

Abstract (EN): The purpose of this study was to compare the performance of the Shell Sort, Insertion Sort, and Quick Sort algorithms across various input sizes and patterns. The three algorithms were implemented in C++, and data were recorded regarding the number of comparisons, recursion depths (where applicable), element movements, and execution times. The study considered arrays of 1 000, 10 000, and 100 000 elements, using five different input patterns, with up to 30 repetitions for each experimental condition. For a dataset of 100 000 elements, initial results revealed significant differences among the algorithms. In scenarios involving duplicates, nearly sorted data, descending order, or random arrangements, Quick Sort yielded lower average execution times than both Shell Sort and Insertion Sort. Conversely, Insertion Sort demonstrated the lowest average execution time for sequences that were already in ascending order. It was preliminarily concluded that the observed performance depends on both the algorithm and the input pattern, which underscores the necessity of accounting for specific experimental conditions and the quality of measurements when conducting such comparisons.

Keywords: Sorting algorithms, Insertion Sort, Shell Sort, Quick Sort, execution time, comparative analysis.

1. INTRODUCCIÓN

Los algoritmos de ordenación posibilitan la organización de datos, lo que simplifica su análisis y búsqueda. El tamaño del arreglo, la distribución de los elementos y las propiedades del algoritmo son factores que determinan el desempeño. Por esta razón, es necesario que su evaluación tenga en cuenta la complejidad computacional y los tiempos de ejecución.

La complejidad algorítmica posibilita el análisis de cómo se incrementa la carga computacional en función del conteo de elementos. A pesar de ello, las mediciones experimentales posibilitan que se aprecie su conducta en situaciones específicas.

La ordenación de los elementos se realiza a través de inserciones sucesivas en el caso del método Insertion Sort, mediante intervalos decrecientes con el método Shell Sort y alrededor de un pivote con el método Quick Sort. Estas disparidades hacen posible que se compare su conducta frente a diferentes dimensiones y patrones de entrada.

El propósito de este estudio fue analizar comparativamente el desempeño de Insertion Sort, Shell Sort y Quick Sort, implementados en C++, con arreglos compuestos por 1 000, 10 000 y 100 000 elementos y cinco patrones diferentes de entrada: aleatorio, en orden ascendente, en orden descendente, casi ordenado y con duplicados. Se anotaron los tiempos de ejecución, las comparaciones, los movimientos y la profundidad más alta de recursión cuando fue necesario. Se formuló la hipótesis descriptiva de que las métricas cambiarían dependiendo del algoritmo, el tamaño del arreglo y el patrón de entrada.

2. MÉTODO

2.1. Algoritmos

Con el fin de comparar la eficiencia de tres algoritmos de ordenamiento desarrollados en C++ (Shell Sort, Quick Sort e Insertion Sort), se llevó a cabo un análisis experimental descriptivo. Sin hacer uso de funciones externas de ordenamiento, se emplearon las implementaciones específicas del proyecto.

Insertion Sort organizó los elementos a través de inserciones sucesivas. Shell Sort utilizó intervalos que disminuían a la mitad en cada fase del proceso. Quick Sort seleccionó como pivote el elemento en el medio del segmento que se estaba procesando y llevó a cabo la ordenación de las particiones mediante llamadas recursivas.

Desde una perspectiva teórica, Insertion Sort presenta una complejidad temporal de $O(n)$ en el mejor de los casos y de $O(n^2)$ en los casos promedio y en el peor de los

casos. Shell Sort tiene una complejidad que depende de la secuencia de intervalos que se aplique. Quick Sort ofrece una complejidad promedio de $O(n\ log\ n)$ y de $O(n^2)$ en el peor escenario. Insertion Sort es un método estable, mientras que Shell Sort y Quick Sort son considerados inestables en las implementaciones evaluadas.

2.2. Generación de Datos y condiciones experimentales

Se evaluaron tres tamaños de entrada: 1 000, 10 000 y 100 000 elementos. Para cada tamaño se consideraron cinco patrones de datos:

- Aleatorio: valores enteros generados entre 0 y 99 999.
- Ascendente: elementos organizados de menor a mayor.
- Descendente: elementos organizados de mayor a menor.
- Casi ordenado: secuencia inicialmente ascendente en la que se intercambiaron posiciones seleccionadas al azar.
- Duplicados: valores enteros generados entre 0 y 9, con elementos repetidos.

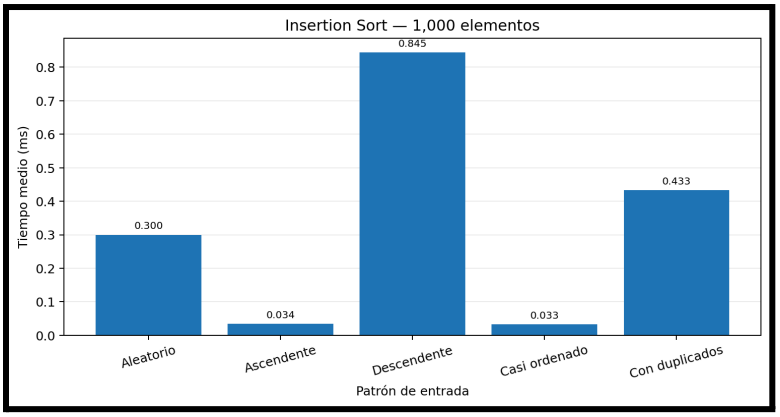
2.3 Entorno computacional y análisis de datos

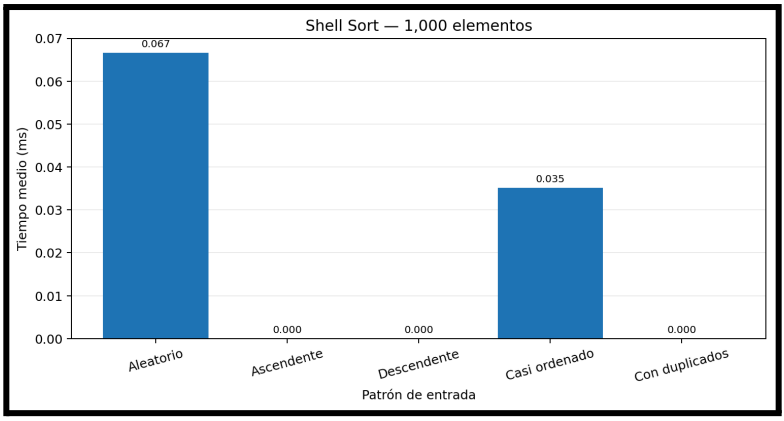
Los experimentos se llevaron a cabo en una máquina con el sistema operativo Windows 10 de 64 bits, con un procesador Intel Core i5 de 7ma generación , una memoria RAM de 16GB y un compilador GCC 14.x mediante MinGW-w64 .

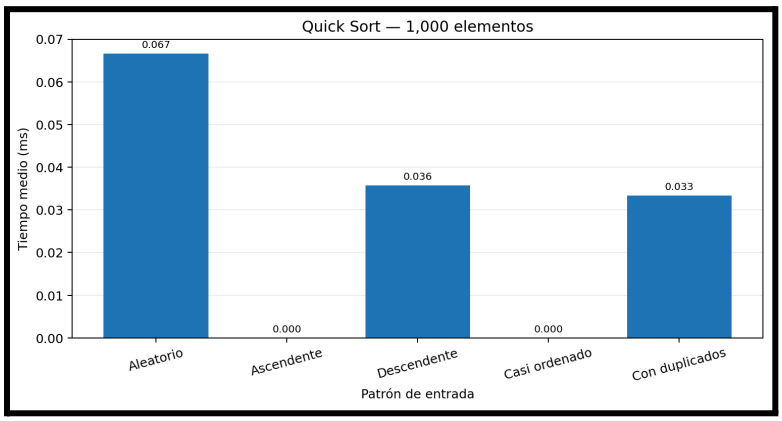
Se emplearon archivos CSV para estructurar las mediciones y analizar el rendimiento de los algoritmos. El análisis estadístico incluyó el cálculo del promedio, la mediana, la desviación estándar, así como los valores mínimo y máximo del tiempo de ejecución para cada combinación de algoritmo, tamaño y patrón. Además, se revisaron las comparaciones, los movimientos y la profundidad de recursión, vinculando las observaciones con la complejidad teórica.

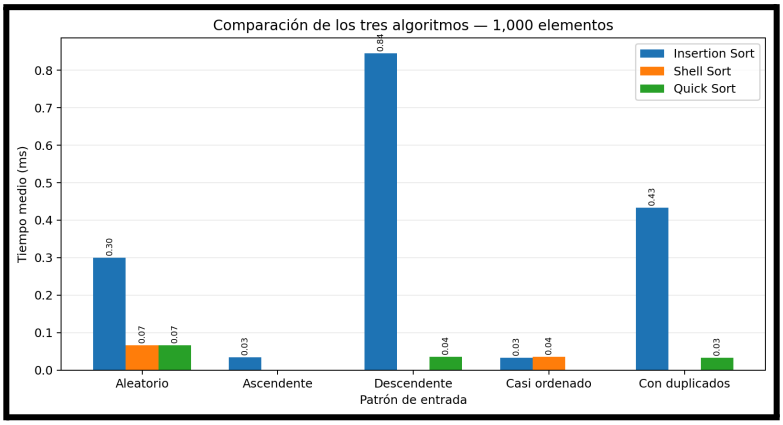
3. RESULTADOS

3.1. (n = 1000)





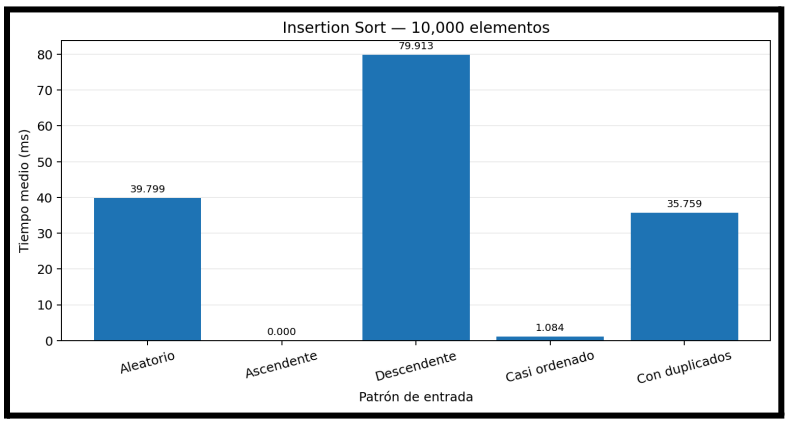


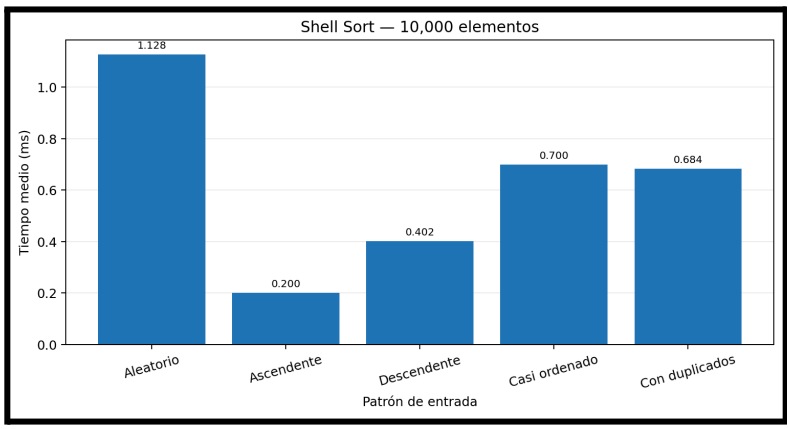


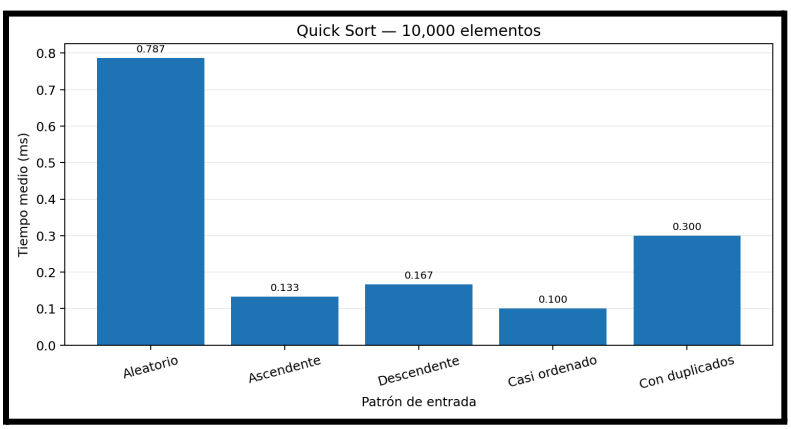
Escenario	Algoritmo	Promedio (ms)	Máximo (ms)	Mínimo (ms)
Aleatorio	Insertion Sort	0.300	1.001	0.000
Aleatorio	Shell Sort	0.067	1.000	0.013
Aleatorio	Quick Sort	0.067	1.000	0.040
Ascendente	Insertion Sort	0.034	0.998	0.017
Ascendente	Shell Sort	0.000	0.000	0.000

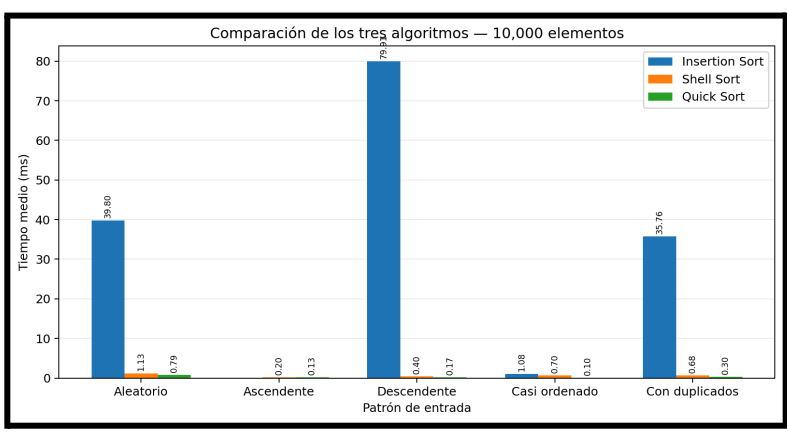
Ascendente	Quick Sort	0.000	0.000	0.000
Descendente	Insertion Sort	0.845	1.503	0.000
Descendente	Shell Sort	0.040	0.000	0.000
Descendente	Quick Sort	0.036	1.000	0.000
Casi ordenado	Insertion Sort	0.033	0.999	0.000
Casi ordenado	Shell Sort	0.035	1.053	0.000
Casi ordenado	Quick Sort	0.000	0.000	0.000
Duplicados	Insertion Sort	0.433	1.002	0.000
Duplicados	Shell Sort	0.000	0.000	0.000
Duplicados	Quick Sort	0.033	0.999	0.000

3.2. (n = 10 000)



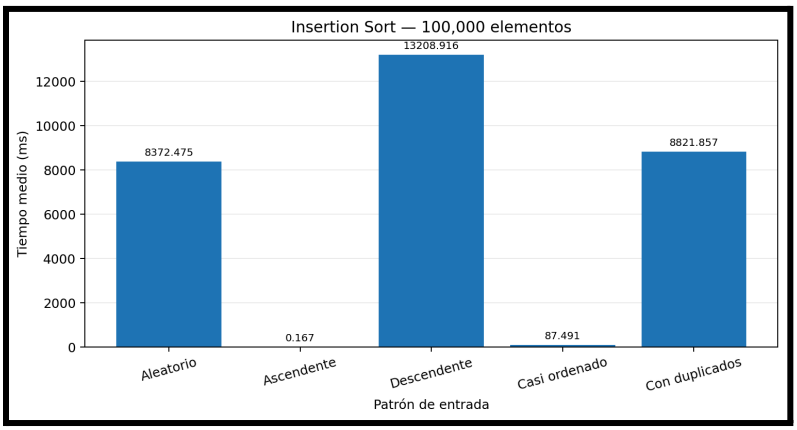


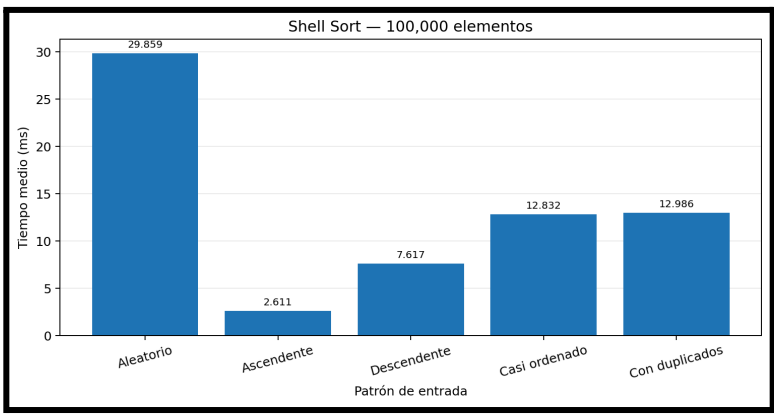


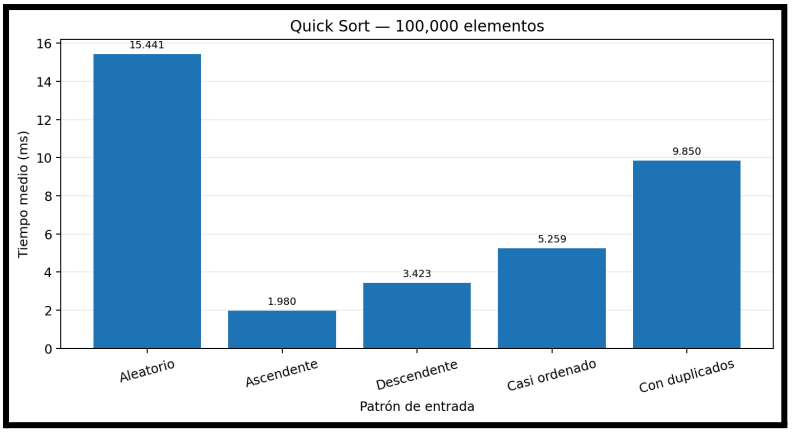


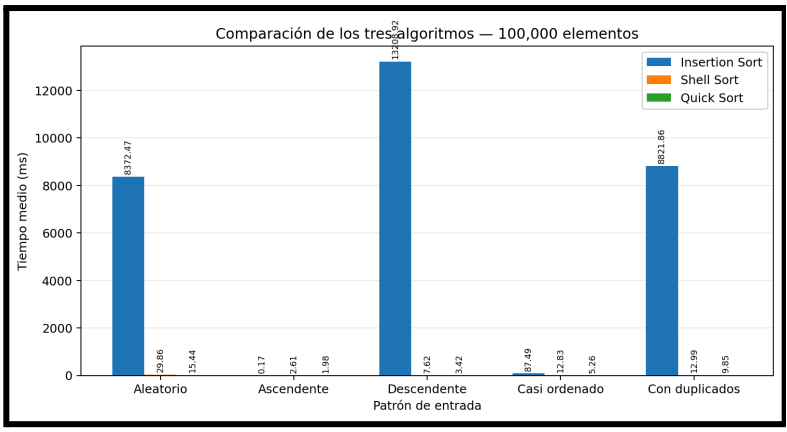
Escenario	Algoritmo	Promedio (ms)	Máximo (ms)	Mínimo (ms)
Aleatorio	Insertion Sort	39.799	42.019	38.506
Aleatorio	Shell Sort	1.128	2.001	0.000
Aleatorio	Quick Sort	0.787	1.003	0.000
Ascendente	Insertion Sort	0.000	0.000	0.000
Ascendente	Shell Sort	0.200	1.002	0.000
Ascendente	Quick Sort	0.133	1.001	0.000
Descendente	Insertion Sort	79.913	82.023	78.516
Descendente	Shell Sort	0.402	1.055	0.000
Descendente	Quick Sort	0.167	1.002	0.000
Casi ordenado	Insertion Sort	1.084	2.000	0.503
Casi ordenado	Shell Sort	0.700	1.002	0.000
Casi ordenado	Quick Sort	0.100	1.000	0.000
Duplicados	Insertion Sort	35.759	37.508	34.506
Duplicados	Shell Sort	0.684	1.502	0.000
Duplicados	Quick Sort	0.300	1.002	0.000

3.3. (n = 100 000)









Escenario	Algoritmo	Promedio (ms)	Máximo (ms)	Mínimo (ms)
Aleatorio	Insertion Sort	8 372.475	11 403.040	3 982.219
Aleatorio	Shell Sort	29.859	40.510	15.505
Aleatorio	Quick Sort	15.441	25.550	7.084
Ascendente	Insertion Sort	0.167	1.000	0.000
Ascendente	Shell Sort	2.611	3.003	1.999
Ascendente	Quick Sort	1.980	2.999	1.000
Descendente	Insertion Sort	13 208.916	21 174.197	7 935.670
Descendente	Shell Sort	7.617	13.609	3.505
Descendente	Quick Sort	3.423	6.004	1.946
Casi ordenado	Insertion Sort	87.491	105.878	81.582
Casi ordenado	Shell Sort	12.832	16.369	11.888
Casi ordenado	Quick Sort	5.259	6.520	4.506
Duplicados	Insertion Sort	8 821.857	9 851.776	3 561.720
Duplicados	Shell Sort	12.986	16.029	5.504
Duplicados	Quick Sort	9.850	12.519	3.505

4. DISCUSIÓN

4.1. Interpretación de los resultados

Los datos evidenciaron diferencias entre Insertion Sort, Shell Sort y Quick Sort. Con 100 000 elementos, Quick Sort obtuvo tiempos medios más bajos en los patrones aleatorios, descendente, casi ordenado y con duplicados. En el patrón ascendente, Insertion Sort alcanzó un menor tiempo medio, aunque conviene comprobar esta medición debido a que se registraron valores de cero milisegundos.

4.2. Relación con la complejidad teórica

Las variaciones observadas guardaron relación con la complejidad temporal de cada algoritmo. En los casos promedio y peor, Insertion Sort tiene una complejidad de $(O(n^2))$, por lo que su tiempo de ejecución puede incrementarse considerablemente a medida que aumenta el tamaño del arreglo. Quick Sort tiene una complejidad promedio de $(O(n\ log\ n))$, mientras que la de Shell Sort está determinada por la secuencia de intervalos empleada. Estas propiedades contribuyen a explicar las diferencias obtenidas en las pruebas.

4.3. Estabilidad de los algoritmos

En la implementación evaluada, Insertion Sort mantuvo el orden relativo de los elementos con valores iguales. Shell Sort y Quick Sort, en cambio, fueron inestables, pues pueden alterar ese orden durante el ordenamiento. Esta propiedad es relevante al trabajar con datos duplicados cuando se requiere conservar el orden original de los elementos equivalentes.

4.4. Limitaciones del estudio

El estudio tuvo algunas limitaciones. Se detectaron registros de cero milisegundos y condiciones en las que se realizaron menos de 30 ejecuciones. Estas dificultades impidieron comparar con precisión algunos resultados. Además, debe tenerse en cuenta que el rendimiento varía según el equipo, el compilador y la implementación empleada. Por esta razón, es necesario verificar las mediciones y completar las ejecuciones pendientes antes de formular conclusiones definitivas.

5. CONCLUSIONES

- El análisis experimental permitió reconocer diferencias de rendimiento entre Insertion Sort, Shell Sort y Quick Sort en función del tamaño y el patrón de entrada. Con 100 000 elementos aleatorios, Quick Sort obtuvo un tiempo medio de 15. 441 ms, mientras que Shell Sort alcanzó 29. 859 ms e Insertion Sort, 8 372. 475 ms.

- En arreglos de 100 000 elementos, Insertion Sort mostró tiempos altos con patrones aleatorios, descendentes y con duplicados. Este resultado concordó con su complejidad temporal cuadrática ($O(n^2)$) en los casos promedio y peor. Por otro lado, su complejidad lineal ($O(n)$) en el mejor caso indica que puede ser eficiente si los datos ya están ordenados, pero esta situación debe corroborarse con las mediciones obtenidas.
- Para 100 000 elementos, Quick Sort alcanzó tiempos medios inferiores a los de Shell Sort e Insertion Sort en cuatro de los cinco patrones analizados. Los resultados concordaron con su complejidad promedio ($O(n\ log\ n)$), aunque su rendimiento puede cambiar de acuerdo con las particiones que se producen durante el ordenamiento.
- La evaluación evidenció, además, que es importante tener en cuenta las características de cada algoritmo. En la implementación utilizada, Insertion Sort fue estable, mientras que Shell Sort y Quick Sort no lo fueron. Esta distinción debe considerarse al ordenar datos con valores repetidos cuando se requiere preservar su orden relativo original.
- Los tiempos registrados como cero milisegundos y las condiciones con menos de 30 ejecuciones redujeron la precisión de algunas comparaciones. Por esta razón, se deben revisar las mediciones y finalizar las pruebas pendientes antes de confirmar los resultados definitivos del estudio.

REFERENCIAS

Hoare, C. A. R. (1962). Quicksort. The Computer Journal, 5(1), 10–16. https://doi.org/10.1093/comjnl/5.1.10

Shell, D. L. (1959). A high-speed sorting procedure. Communications of the ACM, 2(7), 30–32. https://doi.org/10.1145/368370.368387

SW Hosting. (2026, 25 de septiembre). Algoritmos de ordenación con ejemplos en C++. https://www.swhosting.com/es/comunidad/manual/algoritmos-de-ordenacion-con-ejemplos-en-c

Jadhav, S. (2023, 6 de noviembre). Exploring sorting algorithms in C++. Medium. https://medium.com/@jadhavshivam0228/exploring-sorting-algorithms-in-c-b20733adf50e